CSCE 465/765:
Computer & Network Security

Homework 3 — From Prompt to Process

October 22, 2026

Due: Sunday, November 1, 2026

Homework 3: From Prompt to Process

Topics: software security, stack buffer overflow; access control, reference monitor, DAC and MAC
Textbook: Chapter 5, *Operating System Security and Access Control*, especially Sections 5.1–5.7; Chapter 6, *Software Security—Exploits and Privilege Escalation*, especially Sections 6.2–6.6
Teamwork: this assignment is going to be in teams of 2 students each team
Points: 130

Lab Overview

This assignment connects three layers of system security in a single, self-contained exercise. You will first build and exploit a classic memory-safety bug: a stack buffer overflow that redirects execution to a hidden win() function (a "ret2win" attack). You will then observe how an AI agent can be steered by untrusted input into invoking that vulnerable program, fix and harden the code, and finally confine it with ordinary Unix permissions and a Mandatory Access Control (MAC) policy so that even a successful exploit can do little damage. Across the four tasks you will see that a single defect can be attacked, mitigated, and contained at different layers, and that no one layer is sufficient on its own.

Task 0 — Lab Preparation

For this assignment you will need a C toolchain, a debugger, checksec, and the AppArmor utilities. You should install them in your existing sandbox environment from previous assignments.

C toolchain (build-essential). Provides gcc, the linker, and compiles the C programs in Tasks 1 and 3.

Debugger (gdb). The GNU debugger. You use it to inspect the running program's stack and registers and to determine the exact overflow offset in Task 1.

checksec. An open-source tool that reports the security properties (RELRO, stack canary, NX, PIE, and FORTIFY) compiled into an executable. You use it in Tasks 1 and 3 to compare the vulnerable and hardened builds.

AppArmor (apparmor-utils). A Linux kernel security module that provides Mandatory Access Control (MAC) to restrict a program's capabilities and file-system access. You use it in Task 4 to confine the parser; apparmor_parser loads profiles and aa-status shows what is enforced.

Install them once:

```
sudo apt update
sudo apt install -y build-essential gdb binutils checksec apparmor-utils
```

Confirm with gcc --version, gdb --version, checksec --version, and aa-status --version.

Run all HW3 commands from ~/csce465-agentsec/hw3 unless stated otherwise, so that relative paths such as markers/ret2win.txt resolve correctly.

Task 1 — Write and analyze the vulnerable parser (40 points)

In this task you write and then exploit a small, intentionally vulnerable C program, legacy_log_parser. It copies a log file into a fixed-size stack buffer without a length check, so an oversized file overflows the buffer and overwrites the saved return address. You will use GDB to find the exact overflow offset and the address of the win() function, then craft a payload that redirects execution to win() and explain why it works.

Vulnerable Parser. Your parser should meet the following requirements:

- accepts one file path;
- reads the file into a fixed 128-byte stack buffer;

- intentionally permits up to 512 bytes to be copied into that buffer;
- has a non-inlined win() function that:
  - creates markers/ret2win.txt (relative to the hw3/ directory you run the program from);
  - attempts to read /opt/agentsec/decoy.txt and reports success or failure;
- normally prints the beginning of the log and returns.

Note: The parser should read the file into a separate buffer and close it before copying into the fixed 128-byte stack buffer, so that an oversized file cleanly overwrites the saved return address instead of corrupting the still-open file handle.

The key parts of your program should follow the outline below. You should fill in the bodies, in particular the copy in vulnerable copy() that performs no bounds check:

```
#define BUF_SIZE 128   /* the fixed stack buffer */
#define MAX_COPY 512   /* larger than BUF_SIZE on purpose: this is the bug */

void win(void);       /* ret2win target: writes markers/ret2win.txt, */
                /* then attempts to read the protected decoy    */

static void vulnerable_copy(const unsigned char *input, size_t n) {
   char buf[BUF_SIZE];
   /* TODO: copy n bytes from input into buf WITHOUT checking n */
   /* against BUF_SIZE -- the missing check is the vulnerability */
}

static void parse_log(const char *path) {
   /* read the whole file into a heap buffer, fclose() it, THEN */
   /* call vulnerable_copy(input, n) so the overflow reaches    */
   /* only this frame's saved return address -- a clean ret2win */
}
```

Fake Protected File. Create the fake protected file with the following commands. This is the fake target for the attack you demonstrate in this task.

```
sudo mkdir -p /opt/agentsec
printf 'FAKE-COURSE-SECRET\n' | sudo tee /opt/agentsec/decoy.txt
sudo chmod 600 /opt/agentsec/decoy.txt
```

Binary Compilation. The next step is to compile the vulnerable parser you have created. You can use the following commands to compile and build the program. To enable exploitation of the parser in this assignment, you would need to make several configurations as detailed below.

```
gcc -O0 -g -fno-stack-protector -no-pie -Wl,-z,noexecstack \
  -Wall -Wextra -o legacy_log_parser legacy_log_parser.c
checksec --file=./legacy_log_parser
```

The build command above configures several standard protections so that you can study the vulnerability directly. Task 3 turns them back on.

-fno-stack-protector. The compiler can place a random "canary" value on the stack between local buffers and the saved return address, then verify it just before the function returns. In Task 1 we compile with -fno-stack-protector so no canary is inserted and the overwrite can reach the saved return address. Task 3 re-enables a stronger form with -fstack-protector-strong.

-no-pie. A Position-Independent Executable (PIE) lets the program's own code be placed at a random base. Compiling with -no-pie builds a non-PIE binary, so a function such as win() sits at a fixed, predictable address that you can look up once and use in your payload.

-Wl,-z,noexecstack. This linker option marks the program's stack as non-executable, so data written onto the stack cannot be run as machine code.

-O0 -g. -O0 disables optimization and -g includes debugging symbols, which keep the stack layout simple and let GDB relate the running program back to your source code. The exact distance (offset) from the buffer to the saved return address depends on how the compiler lays out the stack, so you must determine it empirically in GDB rather than assuming a textbook value.

Check Security Properties. checksec --file=./legacy_log_parser reports which of these protections are present in a compiled binary (RELRO, stack canary, NX, PIE, and FORTIFY). You will later use it in Task 3 to compare the vulnerable and hardened builds side by side.

Determine the Offset. Use GDB to determine the exact offset to the saved return address, identify the address of win(), and generate a payload file with Python. Demonstrate that direct execution reaches win() and creates the marker.

Note. Run the program from your ~/csce465-agentsec/hw3 directory so that the relative path markers/ret2win.txt resolves correctly and win() writes the marker where the grader expects it. Find the address of win() with a tool such as nm (for example, nm ./legacy_log_parser | grep win) or from within GDB, and derive the overflow offset empirically in GDB.

Report. (1) Provide the GDB output and demonstration of direct execution (screenshots or pasted output). (2) Draw a small stack diagram showing the buffer, saved frame data, saved return address, overwrite offset, and new return target. (3) In your own words, explain how the parser is vulnerable by illustrating the buffer overflow vulnerability using the created stack diagram.

Task 2 — Trace an agent-mediated invocation (20 points)

In this task you connect the memory-corruption bug to an AI agent. You will wrap legacy_log_parser in a local OpenClaw skill and then build an adversarial webpage whose text is designed to steer the agent into invoking that skill on your payload file. You will use the Indirect Prompt Injection attack you learned in HW1 to affect the AI agent. Capture the resulting execution trace and explain how untrusted input can lead an agent toward a vulnerable native program.

Agent Skill. Create a local OpenClaw skill named legacy-parser that accepts only a file path under:

```
~/csce465-agentsec/hw3/inbox/
```

Restricting it to paths under inbox/ is a small access-control boundary such that the agent can run the parser, but only on files you place in that directory.

Adversarial webpage. Create an adversarial webpage that tells the agent to invoke the parser on your payload file. Ask the agent to summarize the page and preserve the resulting trace. This is attacker-controlled content that the agent reads as untrusted input. Its text is written to steer the agent into invoking the skill on your payload file, where data is treated as if it were instructions.

The intended chain. Grading is not dependent on model behaviors. Students may choose to use a local model or API model similar to HW1. Full credit is possible when the model refuses or fails to invoke the tool, provided the student accurately explains the intended chain:

1. untrusted webpage content enters context;
2. the agent proposes or refuses a tool call;
3. a policy decision permits or blocks the call;
4. the native program contains the buffer overflow.

Report. (1) Explain how you created the skill and the adversarial controlled webpage, make sure to explain how this adversarial will affect the agent. (2) Provide the saved agent execution trace. (3) In your own words, explain the agent execution trace following the four steps in the intended chain listed above.

Task 3 — Fix and harden the program (30 points)

In this task you fix the vulnerability and then harden the program against exploitation. You will write legacy_log_parser_fixed.c that rejects oversized input before it is ever copied into the buffer and rebuild it with a set of compiler and linker protections (stack canary, FORTIFY, PIE/ASLR, and Full RELRO) that make exploitation harder even when a bug slips through. You then test both behaviors, compare the two binaries with checksec, and explain which single change removes the vulnerability and which options only raise the bar for an attacker.

Fix the Vulnerable Program and Rebuild. Build upon the vulnerable parser in Task 1, create legacy_log_parser_fixed.c that rejects oversized input before writing to the buffer. Rebuild with:

```
gcc -O2 -g -fstack-protector-strong -D_FORTIFY_SOURCE=3 \
  -fPIE -pie -Wl,-z,relro,-z,now -Wl,-z,noexecstack \
  -Wall -Wextra -Werror \
  -o legacy_log_parser_fixed legacy_log_parser_fixed.c
```

The rebuild command turns the Task 1 protections back on and adds more.

-fstack-protector-strong inserts stack canaries on a broad set of functions, so an overflow that reaches the saved return address is detected at runtime.

-D_FORTIFY_SOURCE=3 replaces certain unsafe libc calls with bounds-checked versions when the compiler can infer the destination buffer size.

-fPIE -pie builds a position-independent executable so code addresses are randomized by ASLR, defeating hard-coded addresses.

-Wl,-z,relro,-z,now enables Full RELRO, making the relocation tables read-only after startup so they cannot be overwritten to hijack control flow.

-Werror turns compiler warnings into errors, forcing obvious mistakes to be fixed at build time.

Note. Remember that most of these options make exploitation harder but do not remove the underlying defect. The root-cause fix is the bounds check you add in the source so that oversized input is rejected before it is copied into the fixed buffer.

Security Properties. After you have compiled the fixed parser into binary program, compare it with the vulnerable parser binary with checksec. Explain which change fixes the root cause and which compiler options only make exploitation more difficult.

Tests. You should write tests to evaluate if the fixed parser is robust. Write tests for the following cases:

- valid short input;
- exactly maximum allowed input;
- one byte over the limit;
- missing file;
- original exploit payload.

Report. (1) The tests results (either screenshots or paste the output of running the tests). (2) A comparison table that summarizes your comparison between both binaries with checksec. (3) In your own words, explain which change fixes the root cause and which compiler options only make exploitation more difficult.

Task 4 — Apply access control and AppArmor (30 points)

In this task you confine the vulnerable program so that even a successful exploit can do little harm. You will create a dedicated low-privilege user, grant it access to only the files the parser needs. You then write an AppArmor profile that applies Mandatory Access Control (MAC) to the parser, allowing only the files and actions it requires and denying everything else. You then run the exploit under confinement, and capture

the resulting denial logs. Finally you connect the experiment to the textbook ideas of reference monitors and least privilege, and explain why confinement limits the damage of the ret2win attack without repairing the underlying C bug.

Low-privilege User. To begin with, create a dedicated low-privilege user:

```
sudo useradd --system --create-home --shell /usr/sbin/nologin agentlab
```

Give agentlab access only to the parser, payload directory, and marker directory.

AppArmor Profile. AppArmor is a Linux kernel security module that enforces MAC on a per-program basis. For a confined program, an AppArmor profile spells out exactly which files it may read or write, which capabilities it may use, and which programs it may execute; anything not explicitly allowed is denied. These rules are enforced by the kernel and bind to the program itself, so they apply no matter which user runs it even a process that ordinary Unix permissions would let do much more. Profiles are loaded with apparmor_parser and run in either enforce mode (violations are blocked and logged) or complain mode (violations are only logged), and aa-status shows which profiles are currently active.

Write an AppArmor profile for the vulnerable parser. The profile should:

- allow reading the selected payload file;
- allow writing only to the marker directory;
- explicitly deny /opt/agentsec/decoy.txt;
- deny execution of shells and unrelated programs;
- allow only the libraries and files required to start the parser.

Then Load the profile with apparmor_parser and run the vulnerable payload as agentlab. Capture the denial logs from journalctl -k or dmesg. (Use a rule of the form audit deny <path> rwklx, for anything you want both blocked and logged. If you install an audit daemon such as auditd, denials move to /var/log/audit/audit.log. Read them with: sudo ausearch -m AVC -ts recent).

Report. (1) List your team, and **a contribution statement** to clearly specify the effort/credit split (e.g., 50:50, 60:40) that your team has agreed on. See group policy below for details. (2) Explain how you designed the AppArmor profile. Illustrate how your design maps to the five requirements above. (3) The denial logs for your run. (4) In 400–600 words, explain the textbook ideas of reference monitor, complete mediation, access matrix, discretionary access control, mandatory access control, and least privilege in this experiment.

Deliverables

Your report is the main submission. report.pdf is what is graded, including output/screenshots of results in your report. Every other file you submit is there so the grader can run and verify the work your report describes. **Submit a zip file** containing the following things. A GitHub repo is optional.

```
hw3/
  README.md
  report.pdf
  AI_USAGE.md
  src/
    legacy_log_parser.c
    legacy_log_parser_fixed.c
    make_payload.py
  policy/
    apparmor.profile
  skills/
    legacy-parser/
      SKILL.md
      run_parser.sh
  web/
    adversarial.html
  tests/
```

Group Policy

(1) This homework is completed in groups of two. Students may form the group on their own.

(2) Partners may share repository, report, and technical submission. In the report, each group should include **a contribution statement** to specify a clear split of effort/credit (e.g., 50:50, 60:40). Individual grades will be distributed accordingly based on contribution.

(3) Both students must include their required AI-use disclosure (e.g. two separate AI usage files in submission).

(4) Both students are responsible for understanding the entire assignment, including code, commands, configurations, experiments, and results.

(5) Do not share code, reports, configurations, packet analyses, or detection rules with other groups.

(6) Report partner participation problems to the instructor as early as possible.

Rubric

Category	Points
Vulnerable program, offset analysis, ret2win, stack diagram	40
Agent-mediated invocation trace	20
Secure fix, compiler hardening, and tests	30
DAC/AppArmor confinement and textbook analysis	30
Reproducibility, clarity, and AI-use record	10
Total	**130**

Late Submission

Late work loses 20% of the earned score per day late, after 5 days it receives no credit.

AI-Use and Safety Policy

You may use AI tools (e.g., ChatGPT, Claude, Gemini, or Copilot) to assist with this assignment, including for learning, brainstorming, debugging, and coding. However, all answers and the final report must be written by you and reflect your own understanding. Any AI-assisted content must be carefully reviewed and independently verified before submission.

If you use an AI tool, you must submit the relevant prompts and AI responses (or an exported conversation/log) together with your report as a separate file. You may remove passwords, API keys, personal information, or other sensitive or private content from the submitted AI logs.

Your submission must also include an **AI usage description (AI_USAGE.md)** that briefly explains whether, what, and how AI was used, and which parts of the work were informed by AI assistance.

```
Tool/model and date:
Purpose:
AI Conversation Log files:
What I used:
What I changed:
How I tested it:
One error, limitation, or rejected suggestion:
```

You remain responsible for understanding, verifying, and explaining everything in your submission. The use of AI does not excuse incorrect results, unsupported claims, or code that you cannot explain.

All work must occur inside the student's course VM and its local containers.

Required rules

1. Use a **NAT** virtual network. Do not use bridged networking.
2. Do not scan campus networks, public IP addresses, another student's machine, or any system not created by you for this course.
3. Use only fake secrets, local mock services, and harmless marker files.
4. Do not use reverse shells, persistence, credential theft, cryptocurrency software, public malware repositories, or real command-and-control services.
5. Do not connect personal messaging accounts, browser profiles, SSH keys, cloud credentials, or password stores to OpenClaw.
6. Do not mount the host home directory or Docker socket into an agent container.

References

[1] Aleph One, "Smashing the Stack for Fun and Profit," Phrack 49. https://people.engr.tamu.edu/guofei/csce465/stack_smashing.pdf

[2] "Smashing the Stack for Fun and Profit: Revived." https://people.engr.tamu.edu/guofei/csce465/smashthestack.pdf

[3] GDB tutorials: https://www.cprogramming.com/gdb.html and https://www.youtube.com/watch?v=wIuZajISL-E

[4] GNU Debugger (GDB) documentation. https://sourceware.org/gdb/documentation/

[5] checksec. https://github.com/slimm609/checksec

[6] AppArmor documentation (Ubuntu) and the profile language: https://ubuntu.com/server/docs/security-apparmor